2.4 ATOMUS: A 5nm 32TFLOPS/128TOPS ML System-on-Chip for Latency Critical Applications

Chang-Hyo Yu, Hyo-Eun Kim, Sungho Shin, Kyeongryeol Bong, Hyunsuk Kim, Yoonho Boo, Jaewan Bae, Minjae Kwon, Karim Charfi, Jinseok Kim, Hongyun Kim, Myeongbo Shim, Changsoo Ha, Wongyu Shin, Jae-Sung Yoon, Miock Chi, Byungjae Lee, Sungpill Choi, Donghan Kim, Jeongseok Woo, Seokju Yoon, Hyunje Jo, Hyunho Kim, Hyungseok Heo, Young-Jae Jin,
Jiun Yu, Jaehwan Lee, Hyunsung Kim, Minhoo Kang, Seokhyeon Choi, Seung-Goo Kim, Myunghoon Choi, Jungju Oh, Yunseong Kim, Haejoon Kim, Sangeun Je, Junhee Ham, Juyeong Yoon, Jaedon Lee, Seonhyeok Park, Youngseob Park, Jaebong Lee, Boeui Hong, Jaehun Ryu, Hyunseok Ko, Kwanghyun Chung, Jongho Choi, Sunwook Jung, Yashael Faith Arthanto, Jonghyeon Kim, Heejin Cho, Hyebin Jeong, Sungmin Choi, Sujin Han, Junkyu Park, Kwangbae Lee, Sung-il Bae, Jaeho Bang, Kyeong-Jae Lee, Yeongsang Jang, Jungchul Park, Sanggyu Park, Jueon Park, Hyein Shin, Sunghyun Park, Jinwook Oh

Rebellions, Seongnam-si, Korea

The growing computational demands of AI inference have led to widespread use of hardware accelerators for different platforms, spanning from edge to the datacenter/cloud. Certain AI application areas, such as in high-frequency trading (HFT) [1-2], have a hard inference latency deadline for successful execution. We present our new AI accelerator which achieves high inference capability with outstanding single-stream responsiveness for demanding service-layer objective (SLO)-based AI services and pipelined inference applications, including large language models (LLM). Owing to low thermal design power (TDP), the scale-out solution can support multi-stream applications, as well as total cost of ownership (TCO)-centric systems effectively.

Latency criticality in AI-as-a-Service is gaining more attention where batching is not always preferrable, especially for a long pipelined service stack and tight SLO services stacks. We have observed that many cloud services hosted in data centers exhibit the input query distributions from single to multiple streams, reflecting the responsive applications and concurrent input assertion use cases. The different service models in public cloud platforms tend to be more focused on responsiveness and stable 99-percentile latency for industrial-automation solutions.

To realize decreased latency [3] and enhanced resource utilization compared to previous AI accelerators [4-7], the proposed ML inference SoC design comprises flexible AI compute cores and a data/control on-chip network in conjunction with latency-centric memory architectures. Fig. 2.4.1 shows the overall SoC architecture of the AI accelerator. The SoC interacts with a host through a PCIe Gen5 interface with single root IO virtualization (SRIOV of 1 physical function and 16 virtual functions) to provide multiple contexts simultaneously, while ensuring higher resource utilization for multi-tenant users within datacenters and maintaining strict isolation between these contexts. ECC-capable DRAM provides high bandwidth data, and additional on-chip reliability features include logic parity and SRAM SECDED. Root-of-trust (RoT) maintains the SoC security in secure booting, firmware updates and runtime integrity checking. From the AI computing perspective, the SoC consists of two neural engine clusters for ML compute connected with a network-on-chip (NoC) data fabric. Each cluster contains quad neural engines of 16TFLOPS/64TOPS ML compute capability, sharing the L1 ML-centric data cache and cluster interconnect for local data transfer operations, such as all-reduce type operations. As separate cores, the neural engines exploit all instruction, data, and thread-level parallelization across dense compute, memory operations and processor-to-processor communications with decentralized RISC-type processor cores within a single engine. The command processor (CP) compiles command descriptor chains proactively for the given computational graphs. It also oversees task scheduling and dynamic control to maximize the utilization of all SoC components and external resources, while maintaining isolation between concurrent contexts through manipulation of page tables across the entire neural engine. The on-chip storage, organized hierarchically as L0 and L2 memories, holds programs and intermediate data for data sharing, enhancing bandwidth while concealing latency within acceptable limits.

Although many in ML community think that peak FLOPS is a good performance proxy, we believe that keeping the computing units busy is one of the most important factors to build a faster yet efficient inference accelerator. Compared to another AI accelerator [4] or inference-target GPUs [5-7], the SoC achieves a latency improvement of 1.4-3.4× for ResNet-50 and performance efficiency (performance over TOPS) of 1.1-5.5× for Bert-Large, as shown in Fig. 2.4.6. To improve the latency and performance efficiency, multi-layered system features with a balanced architecture are employed. Fig. 2.4.2 shows a multiple level synchronization scheme and a hierarchical memory subsystem architecture. Synchronization is one of the critical factors in terms of latency and utilization due to overheads arising from data dependencies. To address the overhead, five levels of synchronization hierarchy are defined. The L0-sync, the lowest level depicted as s0 in Fig. 2.4.2, resolves the intra-core instruction level and the L1-sync (s1) is targeting inter-core synchronization within a cluster. The L2-sync (s2) tackles synchronization over the cluster and the neural task-level DMA engine managed by the task managers (TMs). The CP handles the coarse granularity task level as depicted in s3, synchronization within the SoC, while dealing with the last level synchronization on an IO coherent interconnect with the host reducing software overhead from coherency, s4. To feed the datapath with required data in a timely manner, a five-level memory subsystem hierarchy has been developed. At the lowest level, depicted as d0 in Fig. 2.4.2, a scratch pad is defined with the highest bandwidth and lowest latency in the SoC. The second level, d1, provides data transfer between nearby engines within a cluster, reducing data bandwidth while preserving latency for streaming and systolic data flow. Fig. 2.4.2 shows a neural cluster cache, d2, which ensures high bandwidth and low latency within the cluster. A shared memory, d3, reduces overall bandwidth requirements in the SoC between the clusters and the external memory, d4. To provide higher effective bandwidth, d3 and d4 are fully interleaved over whole channels. Non-shareable traffic to and from the host fully utilizes the peak bandwidth of the protocol through the non-coherent high-bandwidth interconnect between the host and external DRAM, shown in Fig. 2.4.2 as d4-a. To hit the required memory bandwidth from a single neural engine perspective, we exploit neural data traffic characteristics effectively by the proposed hierarchical memory subsystem as shown in Fig. 2.4.3, d1-d4. The necessary memory bandwidth to the external DRAM can be decreased to 6.25% of the port bandwidth from a single neural engine. This is achieved through the local buses, the L1 neural cache, and the L2 shared memory in which the shared memory is extensively interleaved sub-banked modules.

Apart from the system features, the CP proactively orchestrates the incoming commands from the host to the neural computing and DMA engines to maintain utilization as high as possible. Fig. 2.4.4 shows a typical case for the task manipulation. The CP issues commands that are independent of the data loaded from a host memory and runs them in parallel by utilizing multiple levels of sync mechanisms supported by the SoC, as shown in Fig. 2.4.2. DMA operations can commence whenever the data is ready, as is notified by the two types of sync signals (s4) in task and (s3) in instruction granularity as shown in Fig. 2.4.4. The CP generates tasks and dispatches them after resolving the specific dependencies which need to be managed by the CP. Otherwise, the TM of each neural engine handles inter-task dependencies, (s3), as deep as possible by interacting with other TMs on the SoC. To further optimize the sync latency, a dedicated sync bus is defined between the neural engines, (s1) and (s2), with 16ns for the longest path. A programmable load/store processor, shown in Fig. 2.4.1, interacts with other programmable processors with sync instructions resolving the dependencies, s0. The overall chip utilization across the internal neural engines is improved and the measured performance results are shown in Fig. 2.4.6.

We believe that prioritizing system balance and optimizing the utilization of compute resources, on-chip and off-chip memory, as well as bandwidth, is essential for enhancing inference solutions to support scale-up and scale-out capabilities in datacenter and edge computing. Fig. 2.4.5 shows the comprehensive outputs of the proposed system features. Up to 98.9% neural engine utilization was measured, which is attributed to the command processing control loop with the balanced internal storage and the memory subsystem hierarchy, the scratchpad, the inter-core local bus for streaming data with shareable but less locality, and the neural cache for the data with localities along with the bandwidth amplifying shared memory.

As shown in Fig. 2.4.6, the SoC offers the best latency score for ResNet-50 single stream in MLPerf™ v3.0 for low-profile PCIe cards (under 75W TDP). The chip achieves outstanding performance efficiency for the encoder-decoder transformer network, T5-3B [8], which can be seen as a superset network of decoder-only transformers like GPT-3, LLaMA, and others when compared to other AI accelerator/GPU cards. It is worth noting that this result was obtained using the first version of software (driver and compiler) so a performance improvement can be expected the coming months and years based on other competitors' records (riding the typical software optimization curve in the ASIC domain).

Acknowledgement:
The authors wish to convey their profound gratitude for the enthusiastic collaboration and invaluable support received from the entire Rebellions team, all contributors from SEMIFIVE and the exceptional process technology assistance provided by Samsung
Foundry